CS 374: Section A - Homework 6, Problem 2

Anirudh Konidala

October 13, 2026

The flatness of the empty sequence is 0. For indices $1 \le i \le j \le n$, let

$$Mode(i, j) = \max_{c \in \Sigma} |\{p : i \le p \le j \text{ and } A[p] = c\}|,$$

the number of copies of the most frequent character in $A[i..j]$. This is a helper quantity, not a recursive function; the algorithm computes it on the fly.

Recursive function

For $1 \le i \le n+1$, $MaxFlat(i)$ = the maximum flatness of any subsequence of the suffix $A[i..n]$.

We need $MaxFlat(1)$.

Recurrence

$$MaxFlat(i) = \begin{cases} 0 & \text{if } i = n+1 \\ \max\limits_{i \le j \le n} \left(Mode(i,j)^2 + MaxFlat(j+1) \right) & \text{if } i \le n \end{cases}$$

Base case. The only subsequence of the empty suffix $A[n+1..n]$ is empty, with flatness 0.

Idea. Index j is the position in A where the first run of the subsequence ends. That first run may use every copy of one character c in $A[i..j]$, and the best choice of c gives $Mode(i, j)$ copies. The rest of the subsequence comes from $A[j+1..n]$.

The recurrence is at least $MaxFlat(i)$. Let Z be a subsequence of $A[i..n]$ with maximum flatness. Because $i \le n$, the one-character subsequence $A[i]$ has flatness 1, so Z is not empty. Let the first run of Z consist of r copies of the character c, and let j be the position in A of the last character of that run. All r copies lie in $A[i..j]$, so $r \le Mode(i, j)$. The rest Z' of Z is a subsequence of $A[j+1..n]$ that does not start with c (otherwise the first run would be longer). So the runs of Z are the first run followed by the runs of Z', and

$$\text{flatness}(Z) = r^2 + \text{flatness}(Z') \le Mode(i, j)^2 + MaxFlat(j+1).$$

The recurrence is at most $MaxFlat(i)$. Fix $i \le j \le n$, let c be a character with $Mode(i, j)$ copies in $A[i..j]$, and let X be the subsequence of all copies of c in $A[i..j]$. Let Y be a subsequence of $A[j+1..n]$ with flatness $MaxFlat(j+1)$. The concatenation XY is a subsequence of $A[i..n]$. If Y does not start with c, its flatness is $Mode(i, j)^2 + MaxFlat(j+1)$. If Y starts with a run of b

copies of c, that run merges with X, and since $(a+b)^2 \geq a^2 + b^2$ for $a, b \geq 0$, the flatness of XY is at least $\mathit{Mode}(i,j)^2 + \mathit{MaxFlat}(j+1)$. In both cases $\mathit{MaxFlat}(i) \geq \mathit{Mode}(i,j)^2 + \mathit{MaxFlat}(j+1)$.

Example. For $A = [\texttt{A}, \texttt{B}, \texttt{A}, \texttt{L}, \texttt{C}, \texttt{C}, \texttt{L}, \texttt{D}, \texttt{L}, \texttt{G}, \texttt{O}, \texttt{O}, \texttt{D}, \texttt{A}, \texttt{O}]$, the first runs end at $j = 3, 9, 10, 15$, with $\mathit{Mode}(1,3) = 2$ (A), $\mathit{Mode}(4,9) = 3$ (L), $\mathit{Mode}(10,10) = 1$ (G) and $\mathit{Mode}(11,15) = 3$ (O). This gives $4 + 9 + 1 + 9 = 23$, the value in the handout.

Memoization and evaluation order

We store $\mathit{MaxFlat}$ in a one-dimensional array $\mathit{MaxFlat}[1..n+1]$. Entry $\mathit{MaxFlat}[i]$ depends only on entries $\mathit{MaxFlat}[j+1]$ with $j+1 > i$, so we fill the array from $i = n+1$ down to $i = 1$.

For a fixed i, the inner loop runs over $j = i, i+1, ..., n$ in increasing order. It keeps an array $\mathit{Count}[c]$ for $c \in \Sigma$, where $\mathit{Count}[c]$ is the number of copies of c in $A[i..j]$, and a number $m = \mathit{Mode}(i,j)$. Moving from $j-1$ to j adds one copy of $A[j]$ and no other change, so $\mathit{Mode}(i,j)$ is the larger of $\mathit{Mode}(i,j-1)$ and the new value of $\mathit{Count}[A[j]]$.

```
MaxFlatness(A[1..n]):
  MaxFlat[n+1] ← 0
  for i ← n down to 1
    for each c ∈ Σ: Count[c] ← 0
    m ← 0
    MaxFlat[i] ← 0
    for j ← i to n
      Count[A[j]] ← Count[A[j]] + 1
      m ← max{m, Count[A[j]]}        ⟨⟨now m = Mode(i, j)⟩⟩
      MaxFlat[i] ← max{MaxFlat[i], m² + MaxFlat[j+1]}
  return MaxFlat[1]
```

Initializing $\mathit{MaxFlat}[i]$ to 0 does not change the result, because every candidate value $m^2 + \mathit{MaxFlat}[j+1]$ is at least 1.

Running time

The alphabet Σ is fixed, so resetting Count takes $O(|\Sigma|) = O(1)$ time for each i. The inner loop does $O(1)$ work for each of its $n - i + 1$ iterations. The total is $O\left(\sum_{i=1}^{n}(n-i+1)\right) = O(n^2)$, so the algorithm runs in

$$\boxed{O(n^2) \text{ time.}}$$

Sources and collaborators: None.

LLM use: I used Claude (Anthropic) as a tutor for this problem: it helped me understand the problem and guided me, through explanation and questions, toward a correct approach. I wrote and typeset this entire submission myself.